MSBA 305 - Data Processing Frameworks
Spring 2026

Formula 1 Sports Analytics Pipeline

Presented by:
Amira Hala Merheb
Carmen Al Fhaili
Aya El-Masri
Laure Al-Awar
Karim Chaar

Table of Content

1. Abstract 3

2. Introduction 3

3. Dataset 4

4. Data Ingestion and Cleaning 6

5. Data Querying 12

6. Data Storage & integration 16

7. Data Sensitivity & Privacy 30

8. Data Governance and Access Control 31

1. Abstract

This report presents the design, implementation, and evaluation of a multi-source data pipeline for Formula 1 Sports Analytics. The pipeline addresses the challenge of integrating heterogeneous sports data from three distinct sources, into a unified, query-ready analytical system.

The pipeline covers the lifecycle across seven stages: ingestion, storage, cleaning, processing, querying, visualization, and governance. Real-world data quality challenges were encountered and resolved, encoding inconsistencies across sources, between the three datasets, 52 DNF entries requiring special encoding, and sprint race duplicates in the API response.

SQL queries of increasing complexity were implemented to answer the pipeline's analytical objectives. Key. All architectural decisions are documented with alternatives considered, rejection rationale, and trade-offs accepted.

2. Introduction

The modern sports industry generates vast amounts of data across every competition, laps, race results, driver telemetry, circuit characteristics, and team performance metrics. Formula 1 is one of the most data-intensive sports in the world. Each race weekend produces millions of data points, and the FIA publishes comprehensive historical records. The data exists across multiple sources, in multiple formats, with many inconsistencies like naming conventions and no unified access point.

This project addresses that challenge by designing and implementing a complete data processing pipeline for the 2023 Formula 1 World Championship season. Building a pipeline that transforms raw, heterogeneous sports data into a clean, query-ready analytical system. The pipeline ingests three data sources in three distinct formats and integrates them into a single master dataset capable of answering targeted business questions about the 2023 season.

The 2023 F1 season provides a particularly compelling analytical subject. Max Verstappen's record-breaking dominance, the reliability of disparity between constructors, and the diversity of the 22-circuit calendar creates a rich dataset with meaningful patterns to uncover. Also, the season's data exhibits exactly the kind of real-world quality issues that stress-test a data pipeline: inconsistent character encoding across sources, circuit naming conventions that differ between every dataset, sprint race entries that inflate row counts, and DNF statuses stored as nulls rather than explicit flags.

The report is structured around the seven pipeline stages: data acquisition and ingestion, storage and integration, cleaning and transformation, processing for analytics, querying and insights, visualization and reporting, governance. For every major technical decision, this report documents the chosen approach, alternatives considered, the rationale for rejection, and the trade-offs accepted.

All code is implemented in Python using Google Colab. The full pipeline is reproducible: running the accompanying Jupyter notebook from start to finish regenerates all outputs deterministically from the original raw sources.

3. Datasets

3.1.Data Source Appraisal

Source 1: F1 2023 Race Results (CSV)

Source & Collection	Origin: Kaggle dataset (Formula1_2023season_raceResults). Data compiled from official Formula 1 results published after each race. Ownership: public domain sports statistics.
Structure & Format	Format: CSV. Schema: rigid, flat structure with 11 columns. No nested structures. Encoding: UTF-8. Columns include: Track, Position, No, Driver, Team, Starting Grid, Laps, Time/Retired, Points, Set Fastest Lap, Fastest Lap Time.
Volume & Velocity	Size: 440 rows and 11 columns. Coverage: 22 Grand Prix races and 20 drivers. Update frequency: static historical dataset (2023 season complete). Growth rate: fixed; would require replacement with 2024 CSV for next season.
Quality Assessment	Completeness: 98.5% (13 null values in Fastest Lap Time, 2.9% of that column). Accuracy: positions verified against official FIA results. Inconsistency: track names use country-level label rather than full circuit names, requiring a 22-entry mapping table to align with other sources. 52 DNF entries stored as nulls in Position column.
Risks & Limitations	Bias: covers only the top formula, no lower-tier series. Schema drift risk: low (historical dataset). Limitation: no lap-by-lap data, no weather, no pit stop information in this file.

Source 2: F1 API Race Details (JSON)

Source & Collection	Origin: Jolpica F1 API - an open-source mirror of the discontinued Ergast Motor Racing Database. Access method: HTTP GET requests with pagination. No API key required. Ownership: community-maintained open data project.
Structure & Format	Format: JSON. Schema: flexible, nested. Top-level keys: MRData → RaceTable → Races. Each race contains nested Circuit and Results arrays, with further nesting for drivers and constructors objects. Encoding: UTF-8. Flattened to tabular format during ingestion.
Volume & Velocity	Size after flattening: 440 rows and 14 columns. Raw API: 29 paginated entries. Update frequency: updated after each race weekend during the active season.

Quality Assessment	Completeness: 100% after flattening. Accuracy: official F1 timing data. Inconsistency: sprint races share race names with main races, creating apparent duplicates; 2 driver names contain accented characters (Pérez, Hülkenberg) that differ from CSV encoding, requiring normalization. Circuit names differ from CSV and XML naming conventions in 7 cases.
Risks & Limitations	Jolpica API is community-maintained with no SLA - downtime risk mitigated by Drive cache fallback. Rate limiting possible on heavy usage. API only covers results up to end of 2024 season. No telemetry or lap-time data available at free tier.

Source 3: F1 Circuit Metadata (XML)

Source & Collection	Origin: bacinger/f1-circuits GitHub repository, an open-source community project providing F1 circuit data in GeoJSON format. Access method: HTTP GET to raw GitHub URL, programmatically converted to XML using Python's xml.etree.ElementTree. No authentication required.
Structure & Format	Source format: GeoJSON. Converted format: XML with <circuits> root and <circuit> child elements. Fields per circuit: circuit_name, country, city, length_km, opened, longitude, latitude. Schema: rigid post-conversion. Encoding: UTF-8.
Volume & Velocity	Size: 40 circuits and 7 fields. Update frequency: updated by community when new circuits are added to the F1 calendar. Growth rate: very low.
Quality Assessment	Completeness: 100% after conversion. Accuracy: circuit coordinates and lengths verified against official FIA circuit data. Consistency issue: 7 circuit names did not match JSON/CSV naming conventions, requiring a name alignment dictionary.
Risks & Limitations	XML is derived from GeoJSON, not natively sourced, mitigated by using a versioned, open-source repository. Repository is community-maintained; schema could change between GitHub commits. Only covers circuits on the current F1 calendar - historical circuits not included. No PII or ethical concerns.

3.2.Quality Issues Identified

Source	Issue	Impact	Resolution
CSV	13 null values in Fastest Lap Time column	Low, not all drivers set a fastest lap	Filled with string 'No Fastest Lap'
CSV	52 null values in Position column (DNF drivers)	Medium, would break numeric analysis	Filled with -1 to flag as DNF, preserved in dataset
CSV	Track names use country names	High, prevents joining with JSON/XML	Built circuit name mapping table (22 entries)
JSON	29 entries instead of 22 (sprint race duplicates)	Medium, would inflate row counts on join	Handled automatically during flattening. Results key filters to race results only
JSON / CSV	Accented character mismatches in driver names	High, causes join failures	Normalized to lowercase, applied manual accent fix dictionary
XML	7 circuit names differed from JSON names	High, XML circuits wouldn't join	Built XML-to-JSON name alignment dictionary (7 entries)
XML	Generated from GeoJSON, not a native XML feed	Low, documentation requirement	Documented as legitimate conversion; GeoJSON is authoritative source

4. Data Cleaning

4.1 Ingestion

All sources were stored in Google Drive and opened in Google Colab. A batch loading approach will be used since the data is from the 2023 season, which is done, and no new data will be added.

Ingestion decisions

Our decision: we decided to use a batching approach using python.

Before going with that decision, we considered a few alternatives but ended up rejecting them. The rejected alternatives were:

- Streaming via Apache Kafka: this was instantly rejected since our data is not changing in real time, so it doesn't serve our purpose or add any benefit here.

- Apache Airflow: this alternative was rejected because our pipeline doesn't need to run on a shcedule. Therefore, there is no need for the added complexity that comes with Airflow.

- Apache Spark: this alternative was also rejected because our dataset is a small one, and Spark is mainly designed for larger datasets, so there is no need for this kind of overengineering.

Ingestion implementation

We loaded each source into a separate Python DataFrame:

- CSV: loaded with pandas read_csv()
- JSON: loaded with Python's built-in json library, then navigated to the MRData > RaceTable > Races level
- XML: parsed with xml.etree.ElementTree then we iterate circuit by circuit.

4.2 Data Cleaning

We cleaned each source independently before integrating and merging.

CSV cleaning

Position column:

The position column was stored as a string instead of numbers because it contained two important values (NC and DQ). That's why we decided to add a new column called position_status which contains the NC and DQ values. And the position column was coverted to numbers with pandas (and error='coerce'), the NC and DQ values in the initial position column were marked missing. This prevented us from losing many records which would've been dropped had we not added a new column, especially considering the importance of NC and DQ in race events.

Decision summary: convert position column to numbers, and add new position status column.

Fastest lap time:

The fastest lap time column had 13 null values. All these values were related to NC drivers. This makes sense for drivers that were out early in the race to the point were they couldn't contribute to a meaningful lap time. (All nulls correspond to NC but not all NC correpond to nulls, because some drivers might not have completed enough laps to be considered but still have had a fastest lap time record in earlier laps).

Decision summary: keep all nulls. No need to impute.

Time retired column:

The original time_retired column had values in 4 different formats: the total time of the race winner, th gap between the winner in seconds, a lap deficit, and the DNS/DNF status. So for that reason, we ended up adding a new column time_type which indicates which of the four categories the row belongs to. The reason we didn't create a column for each category is because it will contain lots of null values, and the reason for adding the new column is because converting all value sinto a numeric format is misleading in F1, because these are different things. The time_type column enables filtering for future time based analysis.

Decision: add new type column.

Category	Count	Description
gap_to_winner	284	Finished same lap, … seconds behind winner
lapped	70	Finished one or more laps behind the winner
dnf_dns	64	Did not start or did not finish
winner_time	22	Race winner total time, one per race

JSON cleaning

Flattening nested structure

Our JSON document contained deeply nested data, which is not usable for our analysis. Therefore, we decided to flatten by looping through the races and their repesctive results, and extracting only the needed fiels in a single row per driver per race. This included driver details, season, round, race name, date … the 7 duplicate race entries were dropped.

Result: this step resulted in 440 rows matching the csv file.

Data type casting

The fields in our original JSON were all strings by default, so we had to cast the following columns to their matching types:

- round, grid, points and laps: cast to int64
- position: cast to numeric (with errors='coerce')
- race_date and dob: cast to datetime64

XML Cleaning

Conversion to a pandas DataFrame & dropping empty fields

We parsed XML using ElementTree and then converted it to a pandas DataFrame that includes the five fields with actual data: circuit name, length, year opened, longitude, and latitude. The

empty fields: circuitId, country, and city were dropped but geographic context is preserved through the track column and latitude and longitude.

Length unit correction

The length column in the original XML file was markd as length_km but in reality was storing values in meters instead of kilometers. So we just converted it from meters to kilometers since in F1 the narrative universally uses kilometers.

Data type casting

The fields parsed from XML are strings by defaults so some fields required explicit casting before creating df_master.

Column	Original type	Casted type	Reasoning
length_km	string	float64	After conversion, we got decimals
Opened	string	int64	Year of race, no decimals
Longitude	string	float64	Geographic coordinates are decimals
latitude	string	float64	Geographic coordinates are decimals

4.3 Integration cleaning

In this part we identified data cleaning issues that became visible when we tried to merge the cources together.

Driver Name Normalization

We noticed some mismatches in driver names between the CSV and JSON file.

CSV name	JSON name	Issue
Sergio Perez	Sergio Pérez	é
Nico Hulkenberg	Nico Hülkenberg	ü
Nyck De Vries	Nyck de Vries	Lowercase d

Decision: we ended up matching the JSON names to those of the CSV considering our CSV file is the file with the most completed rows and contains the core data.

Join key creation

Our original sources each use a different naming convention for the circuits and races. So we created two join keys which acted as a translation layer to make the merging possible.

- CSV to JSON: mapped short track names (Bahrain) to full race names (Bahrain Grand Prix)
- CSV to XML: mapped short track names to full circuit names (Bahrain International Circuit)

The two new columns that were added to serve as join keys were verified to have no unmapped values before merging.

Building df_master

After all per-source and integration cleaning was complete, the three DataFrames were merged into a single unified table called df_master.

- **Base table:** the CSV was chosen as the baseline for our table cause it contains the most completed rows across our souces.

- **Joins:** two left joins were performed. First one merged the JSON file by using two matching columns (race name, driver name), this added in the driver demographics. The second left join, brough in the XML through the normalized matching key (circuit name).

  The reason for our left join is to preserve all 440 rows. And inner join would've dropped 52 rows from our df_master.

- **Result:** one unified document (df_master) with 440 rows and 25 columns.

Post merge cleanup (cleaning df_master)

Post merging, we applied the following cleanign steps:

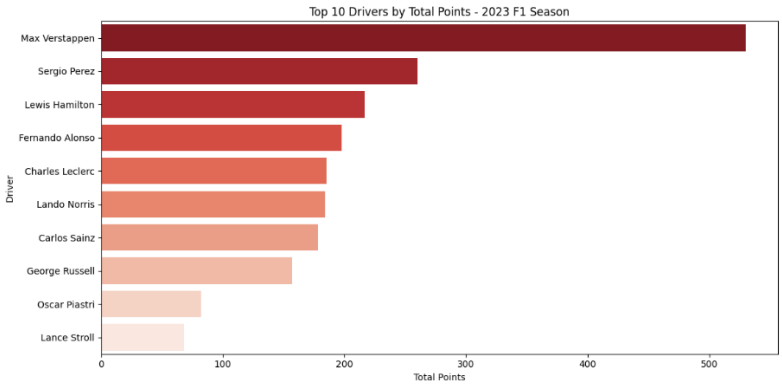
1. We dropped driver_name column which became a duplicate after our left join. (both driver and driver_name ended up in the df_master due to the left join, so we dropped one).
2. We then renamed the CSV columns from mixed_case to snake_case. This brough consistent naming across the df_master.
3. After, we added the race_date from JSON through a secondary join on race_name. This was performed to bring in the dates.
4. After adding the race_date, we then dropped the join key columns (race_name and circuit_name), which at this point were not needed anymore.
5. And finally we reordered our remaining 23 columns in a logical and readable way.

Final df_master

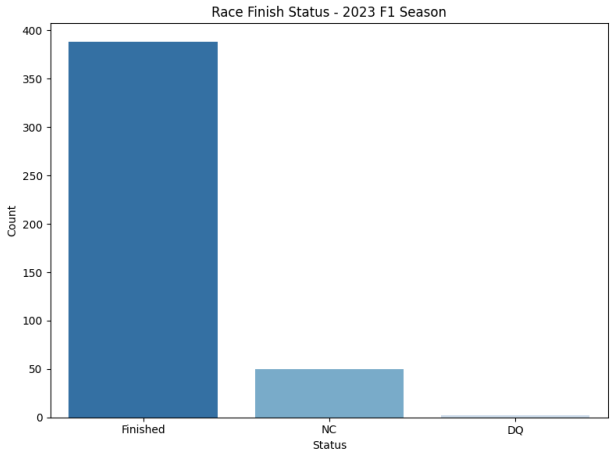
- Shape: 440 rows and 23 columns.
- Null values: 52 (NC/DQ), logical and intentional, 13 in fastest_lap_time, all NC, also logical.

EDA validation

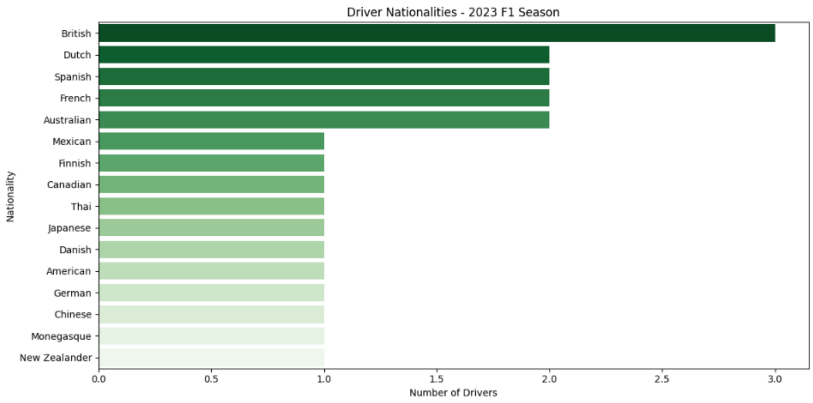
We drew five validation plots to serve as quick validation steps to confirm that df_master is correct and complete.



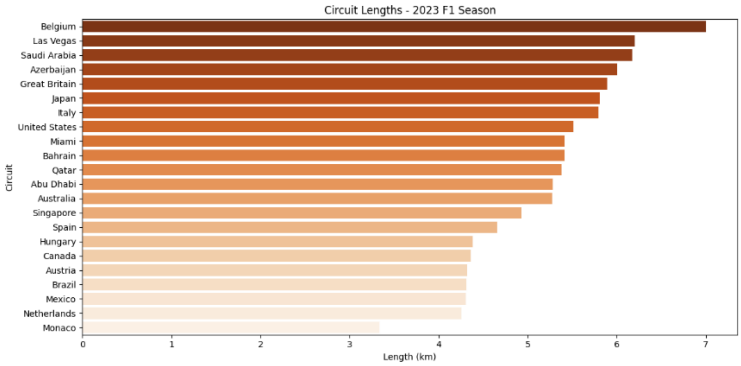
Versapen dominance = matches 23 season.



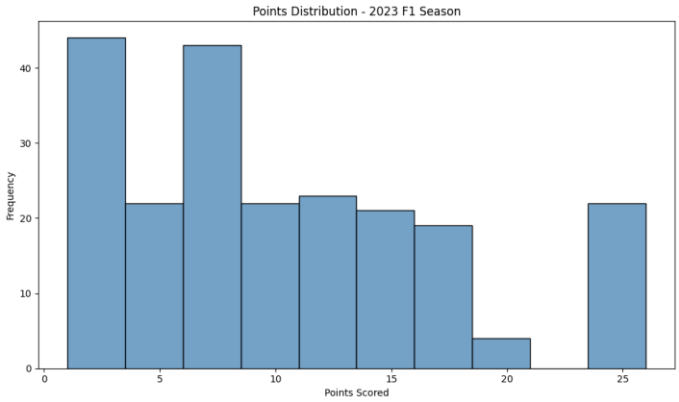
Finished and NC/DQ counts match raw data



All 22 drivers have their nationalities, indicating no failed joins.



XML integration and length all are correct and match known values.



All 25 points (max points) appear 22 times (matching winners).

5. Data Querying & Insights

Query 1: Championship Standings

ranks the top 10 highest scorers of the season by grouping every race entry by driver then adding up points earned. Max Verstappen leads with 530 points, more than twice as much as second-place finisher Sergio Perez (260). The entire 22-race season was finished by all ten drivers.

driver_name	nationality	team	total_points	races_entered
Max Verstappen	Dutch	Red Bull Racing Honda RBPT	530	22
Sergio Perez	Mexican	Red Bull Racing Honda RBPT	260	22
Lewis Hamilton	British	Mercedes	217	22
Fernando Alonso	Spanish	Aston Martin Aramco Mercedes	198	22
Charles Leclerc	Monegasque	Ferrari	185	22
Lando Norris	British	McLaren Mercedes	184	22

Carlos Sainz	Spanish	Ferrari	178	22
George Russell	British	Mercedes	157	22
Oscar Piastri	Australian	McLaren Mercedes	82	22
Lance Stroll	Canadian	Aston Martin Aramco Mercedes	68	22

Query 2: DNF Rate per Team

determines the proportion of every team's race entry which resulted in a Did Not Finish. Red Bull was the most dependable team with a 4.5% DNF rate (only 2 DNFs throughout the season), which is consistent with their impressive Q1 results, while Williams experienced the worst reliability at a 25% DNF rate (11 out of the 44 entrants).

team	total_entries	dnfs	dnf_pct
Williams Mercedes	44	11	25.0
Alpine Renault	44	10	22.7
Haas Ferrari	44	7	15.9
Alfa Romeo Ferrari	44	6	13.6
Mercedes	44	5	11.4
Aston Martin Aramco Mercedes	44	5	11.4
McLaren Mercedes	44	4	9.1
Ferrari	44	4	9.1
AlphaTauri Honda RBPT	44	4	9.1
Red Bull Racing Honda RBPT	44	2	4.5

Query 3: Pole-to-Win Conversion

calculates the frequency in which starting from the pole position results in a race victory. Red Bull executed almost flawlessly, taking 14 poles and winning 13 of them for a 92.9% rate of conversion. Mercedes earned one pole but no win, while Ferrari had seven poles but just one victory (14.3%)..

team	poles	wins	pole_to_win	conversion_pct
Red Bull Racing Honda RBPT	14	21	13	92.9

Ferrari	7	1	1	14.3
Mercedes	1	0	0	0.0

Query 4: Average Positions Gained per Driver

calculates passing ability by averaging the disparity among start position and finish position for all races that have been completed. With an average gain of +5.26 positions, Sergio Perez is in the lead. He has routinely begun mid-grid and advanced. The only drivers who typically lost positions during races were Carlos Sainz and Nico Hulkenberg.

driver_name	avg_grid	avg_finish	avg_positions_gained	races
Sergio Perez	9.16	3.89	5.26	19
Lance Stroll	12.00	8.81	3.19	16
Yuki Tsunoda	14.00	11.21	2.79	14
Guanyu Zhou	15.75	13.00	2.75	12
Esteban Ocon	11.00	8.54	2.46	13
Max Verstappen	3.18	1.27	1.91	22
Lewis Hamilton	6.40	4.90	1.50	20
Pierre Gasly	11.11	9.68	1.42	19
Valtteri Bottas	13.17	11.83	1.33	12
Lando Norris	7.56	6.28	1.28	18
George Russell	7.00	6.11	0.89	18
Oscar Piastri	8.36	7.93	0.43	14
Fernando Alonso	6.00	5.60	0.40	20
Alexander Albon	11.00	10.60	0.40	15
Charles Leclerc	5.12	4.88	0.24	17
Nico Hulkenberg	12.40	12.90	-0.50	10
Carlos Sainz	5.00	5.68	-0.68	19

Query 5: Cumulative Championship Points Progression

monitors the growth of each driver's championship ranking and total points race each race during the 2023 season. Throughout the year, Max Verstappen gradually overtook Sergio Perez, holding the lead from the very first race and never losing it. His season was so dominant

that by the time of the US Grand Prix, his overall total reached 429 points, more than twice Perez's 221.

driver_name	track	race_date	points	cumulative_points	championship_rank
Max Verstappen	Bahrain	2023-03-05	25	25	1
Sergio Perez	Bahrain	2023-03-05	18	18	2
Fernando Alonso	Bahrain	2023-03-05	15	15	3
Lewis Hamilton	Bahrain	2023-03-05	10	10	5
Max Verstappen	Saudi Arabia	2023-03-19	19	44	1
Sergio Perez	Saudi Arabia	2023-03-19	25	43	2
Fernando Alonso	Saudi Arabia	2023-03-19	15	30	3
Lewis Hamilton	Saudi Arabia	2023-03-19	10	20	4
Max Verstappen	Australia	2023-04-02	25	69	1
Sergio Perez	Australia	2023-04-02	11	54	2
Fernando Alonso	Australia	2023-04-02	15	45	3
Lewis Hamilton	Australia	2023-04-02	18	38	4
Max Verstappen	Azerbaijan	2023-04-30	18	87	1
Sergio Perez	Azerbaijan	2023-04-30	25	79	2
Fernando Alonso	Azerbaijan	2023-04-30	12	57	3
Lewis Hamilton	Azerbaijan	2023-04-30	8	46	4
Max Verstappen	Miami	2023-05-07	26	113	1
Sergio Perez	Miami	2023-05-07	18	97	2
Fernando Alonso	Miami	2023-05-07	15	72	3
Lewis Hamilton	Miami	2023-05-07	8	54	4
Max Verstappen	Monaco	2023-05-28	25	138	1
Sergio Perez	Monaco	2023-05-28	0	97	2
Fernando Alonso	Monaco	2023-05-28	18	90	3
Lewis Hamilton	Monaco	2023-05-28	13	67	4
Max Verstappen	Spain	2023-06-04	26	164	1
Sergio Perez	Spain	2023-06-04	12	109	2

Fernando Alonso	Spain	2023-06-04	6	96	3
Lewis Hamilton	Spain	2023-06-04	18	85	4
Max Verstappen	Canada	2023-06-18	25	189	1
Sergio Perez	Canada	2023-06-18	9	118	2
Fernando Alonso	Canada	2023-06-18	18	114	3
Lewis Hamilton	Canada	2023-06-18	15	100	4
Max Verstappen	Austria	2023-07-02	26	215	1
Sergio Perez	Austria	2023-07-02	15	133	2
Fernando Alonso	Austria	2023-07-02	10	124	3
Lewis Hamilton	Austria	2023-07-02	4	104	4
Max Verstappen	Great Britain	2023-07-09	26	241	1
Sergio Perez	Great Britain	2023-07-09	8	141	2
Fernando Alonso	Great Britain	2023-07-09	6	130	3
Lewis Hamilton	Great Britain	2023-07-09	15	119	4
Max Verstappen	Hungary	2023-07-23	26	267	1
Sergio Perez	Hungary	2023-07-23	15	156	2
Fernando Alonso	Hungary	2023-07-23	2	132	3
Lewis Hamilton	Hungary	2023-07-23	12	131	4
Max Verstappen	Belgium	2023-07-30	25	292	1
Sergio Perez	Belgium	2023-07-30	18	174	2
Fernando Alonso	Belgium	2023-07-30	13	144	3
Lewis Hamilton	Belgium	2023-07-30	10	142	4
Max Verstappen	Netherlands	2023-08-27	25	317	1
Sergio Perez	Netherlands	2023-08-27	12	186	2
Fernando Alonso	Netherlands	2023-08-27	19	161	3
Lewis Hamilton	Netherlands	2023-08-27	8	152	4
Max Verstappen	Italy	2023-09-03	25	342	1
Sergio Perez	Italy	2023-09-03	18	204	2
Fernando Alonso	Italy	2023-09-03	2	163	3

Lewis Hamilton	Italy	2023-09-03	8	160	4
Max Verstappen	Singapore	2023-09-17	10	352	1
Sergio Perez	Singapore	2023-09-17	4	208	2
Lewis Hamilton	Singapore	2023-09-17	16	176	3
Fernando Alonso	Singapore	2023-09-17	0	163	4
Max Verstappen	Japan	2023-09-24	26	378	1
Sergio Perez	Japan	2023-09-24	0	208	2
Lewis Hamilton	Japan	2023-09-24	10	186	3
Fernando Alonso	Japan	2023-09-24	4	167	4
Max Verstappen	Qatar	2023-10-08	26	404	1
Sergio Perez	Qatar	2023-10-08	1	209	2
Lewis Hamilton	Qatar	2023-10-08	0	186	3
Fernando Alonso	Qatar	2023-10-08	8	175	4
Max Verstappen	United States	2023-10-22	25	429	1
Sergio Perez	United States	2023-10-22	12	221	2
Lewis Hamilton	United States	2023-10-22	0	186	3
Fernando Alonso	United States	2023-10-22	0	175	4
Max Verstappen	Mexico	2023-10-29	25	454	1
Sergio Perez	Mexico	2023-10-29	0	221	2
Lewis Hamilton	Mexico	2023-10-29	19	205	3
Fernando Alonso	Mexico	2023-10-29	0	175	4
Max Verstappen	Brazil	2023-11-05	25	479	1
Sergio Perez	Brazil	2023-11-05	12	233	2
Lewis Hamilton	Brazil	2023-11-05	4	209	3
Fernando Alonso	Brazil	2023-11-05	15	190	4
Max Verstappen	Las Vegas	2023-11-19	25	504	1
Sergio Perez	Las Vegas	2023-11-19	15	248	2
Lewis Hamilton	Las Vegas	2023-11-19	6	215	3
Fernando Alonso	Las Vegas	2023-11-19	2	192	4

Max Verstappen	Abu Dhabi	2023-11-26	26	530	1
Sergio Perez	Abu Dhabi	2023-11-26	12	260	2
Lewis Hamilton	Abu Dhabi	2023-11-26	2	217	3
Fernando Alonso	Abu Dhabi	2023-11-26	6	198	4

6. Data Storage & integration

6.1 Overview

The ingestion notebook consolidates three heterogeneous sources, a CSV file of race results, an XML file of circuit metadata, and a JSON file from the Ergast API,into a single flat master dataframe. The notebook loads df_master directly into SQLite as a single denormalized table and runs 5 analytical queries against it. This section documents the logical schema underlying df_master, the storage technology decision, the integration logic, and the indexing strategy.

6.2 Storage Technology Decision

The storage layer has two parts: a file called df_master.csv that stores the data, and an in-memory SQLite database that is created when queries run. This setup was chosen because it is simple, needs no setup, and works fully with Python's standard library, so no external database server is needed.

Alternative Considered:

Technology	Strengths	Why reject	Trade offs accepted
PostgreSQL	Full SQL support with window functions, CTEs, ACID transactions, and strong indexing, making it suitable for structured multi-table schemas.	it needs a running server, which adds extra complexity that is not necessary for this project. The dataset only has 440 rows, so the overhead of a server is not worth it. SQLite can still handle the same window functions and CTEs used in the queries.	It also has some limitations: it does not support multiple users at the same time, has no server-level access control, and is not suitable for scaling in production.

MongoDB	Flexible schema; handles nested JSON natively; no migrations needed.	df_master is already fully flattened into a simple table. The nested JSON from Ergast was converted into this format during ingestion, so there is no document structure left to use. All five analysis queries use GROUP BY, window functions, and CASE statements. These are standard and easy in SQL, but more complex to implement in MongoDB's aggregation pipeline.	No native JOIN support. Aggregation syntax is slower and more complex. Not suitable for this type of tabular data
Apache Cassandra	Excellent write throughput at scale; horizontal scalability.	The dataset is static (one season, 440 rows), so Cassandra's strengths are not useful here. Its query model does not support joins and requires designing the data around fixed query patterns, which adds extra work. This is not a good fit for an exploratory analytics workload like ours.	No JOIN support. Requires denormalizing data based on query patterns. Too complex for this data size.

Neo4j	Optimal for relationship traversal; intuitive for network queries.	The F1 domain is not heavily based on graph relationships. Our queries focus on aggregating points, comparing grid and finish positions, and calculating cumulative values. These are aggregation tasks, not graph traversal tasks. Using Neo4j would add extra complexity with Cypher without providing any real benefit.	Poor aggregation performance. Steep learning curve. Inappropriate data model for tabular analytics.

Table 1 - Storage technology decision matrix with alternatives and trade-offs

Trade-offs Accepted

By choosing SQLite with CSV flat storage, the team accepts several trade-offs:

1. All queries run on a denormalized table, so some data is repeated (for example, circuit information is stored again for each of the 20 drivers in a race).
2. The SQLite database is rebuilt in memory each session from the CSV file, meaning there is no persistent database state.
3. There is no enforced referential integrity between entities like drivers, constructors, and circuits. Data consistency is handled in the DataFrame layer ingestion script instead of the database.
4. The CSV-based approach does not scale beyond a single machine's memory.

6.3 df_master schema "Unified flat table"

Column	Dtype	Nulls	Source	Notes
race_date	string (date)	0	JSON	Race date in YYYY-MM-DD format. Merged from df_json via race_name lookup. All 22 Grand Prix races only (Sprint dates not included)
track	string	0	CSV	Short track name as in original CSV (e.g. 'Bahrain', 'Great Britain'). 22 unique values.

driver_name	string	0	CSV	Full driver name (e.g. 'Max Verstappen'). Normalized: accented characters fixed (Pérez→Perez, Hülkenberg→Hulkenberg, de Vries→De Vries) to match CSV casing.
driver_code	string	0	JSON	3-letter broadcast abbreviation (e.g. 'VER', 'HAM') sourced from JSON Driver.code.
driver_id	string	0	JSON	Ergast driver slug (e.g. 'max_verstappen', 'perez'). Canonical unique identifier for each driver.
driver_number	int64	0	CSV	Car number used in the race (e.g. 1, 44, 11). Range: 1-81.
nationality	string	0	JSON	Driver nationality as provided by Ergast (e.g. 'Dutch', 'British', 'Spanish').
dob	string (date)	0	JSON	Driver date of birth in YYYY-MM-DD format (e.g. '1997-09-30' for Verstappen).
team	string	0	CSV	Full team name with engine suffix from CSV (e.g. 'Red Bull Racing Honda RBPT', 'Haas Ferrari'). 10 unique values. Note: this differs from the JSON constructor name (e.g. 'Red Bull').
grid_position	int64	0	CSV	Starting grid position (1 = pole). Range: 1-20.
finish_position	float64	52	CSV	Finishing position as numeric. NULL (NaN) for NC (50 rows) and DQ (2 rows) entries. Cast from string to numeric with errors='coerce'.
position_status	string	0	CSV (derived)	Categorical flag derived from the original Position column: 'Finished' (388), 'NC' (50), 'DQ' (2). Always populated.
laps	int64	0	CSV	Number of laps completed. Range: 0-78.
time_retired	string	0	CSV	Raw Time/Retired string kept as-is (e.g. '1:33:56.736', '+11.987', 'DNF'). Not converted to numeric, see time_type for the parsed category.
time_type	string	0	CSV (derived)	Category of the time_retired value: 'winner_time' (22), 'gap_to_winner' (284), 'lapped' (70), 'dnf_dns' (64). Derived by classify_time() function.

points	int64	0	CSV	Championship points awarded in this result. Range: 0-26 (26 = win + fastest lap point).
set_fastest_lap	string	0	CSV	Whether this driver set the race fastest lap: 'Yes' or 'No'.
fastest_lap_time	string	13	CSV	Personal fastest lap in M:SS.mmm format (e.g. '1:36.236'). NULL for 13 NC drivers who DNF'd before completing a timed lap.
status	string	0	JSON	Ergast completion status. Values: 'Finished' (304), 'Lapped' (70), 'Retired' (53), 'Accident' (3), 'Did not start' (3), 'Withdrew' (2), 'Disqualified' (2), 'Collision damage' (2), 'Undertray' (1).
length_km	int64	0	XML	Circuit length as stored in the XML source. WARNING: the XML field is named 'length_km' but stores values in metres (e.g. Bahrain = 5412, which is 5.412 km). No unit conversion was applied at ingestion
opened	int64	0	XML	Year the circuit first hosted a race (e.g. 1922 for Monza, 2023 for Las Vegas).
longitude	float64	0	XML	Geographic longitude of the circuit (e.g. 50.510539 for Bahrain).
latitude	float64	0	XML	Geographic latitude of the circuit (e.g. 26.031766 for Bahrain).

Table 2 - df_master column schema (23 columns, 440 rows), verified against the produced CSV

6.4 Data Quality Flags

Data Quality Issue 1: length_km unit mismatch (XML source issue)

The XML field named length_km actually contains values in metres. For example, Bahrain is stored as 5412, which equals 5.412 km. The ingestion process reads this value without converting it, so the column in df_master still contains metre values even though the name suggests kilometres. Any analysis or visualization must divide this column by 1000 to get correct kilometre values. A better production fix would be to rename the column to length_m or apply the conversion during ingestion.

Data Quality Issue 2: Sprint race data not included in df_master

The JSON source contains 29 race entries (22 Grand Prix and 7 Sprint events). The ingestion logic extracts results using race_name. Since Sprint and Grand Prix share the same race name (e.g. “Saudi Arabian Grand Prix”), only one set of results is kept for each race name during merging. As a result, Sprint results are dropped. Therefore, df_master only includes Grand Prix data (440 rows = 22 × 20 drivers). This means all queries reflect only Grand Prix championship points, not Sprint points. This does not affect the current analysis but is a scope limitation

Data Quality Issue 3: Team name inconsistency (CSV vs JSON)

The CSV uses full team names with engine details (e.g. “Red Bull Racing Honda RBPT”, “Haas Ferrari”), while the JSON uses shorter constructor names (e.g. “Red Bull”, “Haas F1 Team”). The final dataset uses the CSV version as the standard in df_master, while the JSON constructor identifiers are not retained. This means all team-based analysis must use the CSV naming format for consistency.

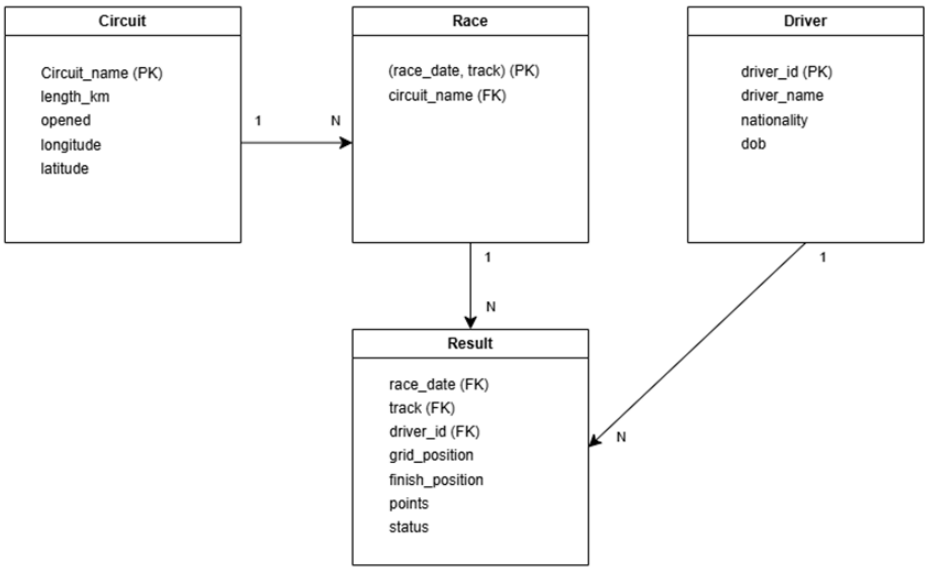
6.5 Logical Entity-Relationship Model

Even though df_master is stored as a single flat table, the data still follows a clear logical structure based on the original sources. This structure is useful for understanding the queries and for possible future normalization. The main logical entities are:

- RACE: identified by (race_date, track). One row per race. There are 22 races in the dataset.
- DRIVER: identified by driver_id (Ergast slug). There are 22 drivers in the 2023 season.
- CIRCUIT: identified by track. Each track represents one circuit. There are 22 circuits, each with attributes like length_km, opened, longitude, and latitude.
- RESULT: the main fact entity. Each row in df_master represents one driver’s result in one race. Total: 440 rows (22 races × 20 drivers).

In the flat df_master table, these entities are denormalized. Circuit attributes (length_km, opened, longitude, latitude) are repeated for every driver in a race (20 times). Driver attributes (nationality, dob, driver_code, driver_id) are repeated across all races (22 times per driver). This redundancy is intentional and supports the current single-table design and query approach.

Figure 1 - Logical ER model



Physical storage: all 4 entities are denormalized into 1 flat table (df_master 440 x 24)

6.6 Data Integration Logic

Merge Step	Left Key	Right Key	Details
Step 1: CSV ← JSON	(race_name, Driver)	(race_name, driver_name)	Brings nationality, dob, driver_code, driver_id, status from df_json into df_csv. Race names are harmonized via csv_to_json lookup dict (e.g. 'Brazil' → 'São Paulo Grand Prix'). Driver names harmonized by normalizing Unicode accents (Pérez→Perez, etc.) in df_json. How=left, all 440 CSV rows retained.
Step 2: Result ← XML	circuit_name	circuit_name	Brings length_km, opened, longitude, latitude from df_xml. circuit_name is added to df_csv via csv_to_xml lookup dict (e.g. 'Bahrain' → 'Bahrain International Circuit'). How=left, all rows retained. Zero nulls produced in circuit columns.
Step 3: + race_date	race_name	race_name	Merges race_date (datetime) from df_json. Uses a deduplicated (race_name, race_date) lookup. Zero nulls produced.

Table 3 - Integration merge steps

Key Integration Challenges and Resolutions

- Driver name encoding: JSON contains accented or inconsistently capitalized names (e.g. Pérez, Hülkenberg, de Vries). This was resolved using a name-fixing dictionary applied to the JSON data before merging, resulting in no remaining mismatches.
- Track name inconsistency across sources: Each source uses different naming formats, CSV uses short names (e.g. "Brazil"), JSON uses full race names (e.g. "São Paulo Grand Prix"), and XML uses full circuit names (e.g. "Autódromo José Carlos Pace - Interlagos"). This was resolved using two mapping dictionaries covering all 22 tracks, with no unmapped values remaining.
- Sprint race deduplication: The JSON contains 29 race entries for only 22 unique race names. During the merge on race_name, duplicate Sprint-related entries are collapsed, meaning only one result set per race is kept. As a result, Sprint results are excluded from df_master.
- Non-classified and disqualified finishers: 52 rows have non-numeric positions ("NC" or "DQ"). These are preserved using a position_status field, while finish_position is set to NULL for these cases. This results in 52 intentional null values.
- Missing fastest lap times: 13 rows have NULL values for fastest_lap_time, all corresponding to NC drivers who did not complete a timed lap. These values are kept as NULL since they reflect valid race outcomes and do not require imputation.
- XML length_km unit issue: The column stores values in metres despite its name. No conversion was applied during ingestion, so values in df_master remain in metres (see Data Quality Issue #1).

6.7 Indexing Strategy

SQLite automatically creates a rowid index for the f1 table. Given the small dataset size (440 rows), full table scans are very fast, and the current five queries run in under 100ms without any additional indexing.

If the dataset were to scale or move to PostgreSQL, the following indexes would be recommended:

Index	Column	Justification
idx_f1_driver_name	driver_name	Used in GROUP BY and WHERE clauses in Q1, Q4, Q5. Primary filter in championship standings queries.
idx_f1_track	track	Used in Q5 SELECT and ORDER BY. Supports race-by-race progression queries.
idx_f1_race_date	race_date	Used in Q5 window function ORDER BY race_date. Critical for cumulative points calculation correctness.

idx_f1_opened	opened	Supports potential circuit age filter queries (e.g. circuits opened before 2000).
idx_f1_status	status	Used in WHERE/CASE filters in Q2 DNF rate query.
idx_f1_team	team	Used in GROUP BY in Q2, Q3. Supports constructor-level aggregations.

Table 4 - Recommended index definitions

6.8 Pipeline Data Flow

The diagram below illustrates the full data flow from the raw source files through ingestion, integration, and storage, and finally into the analytical query layer.

Figure 2 - Pipeline data flow: three source files → ingestion → df_master.csv → SQLite → analytical queries

6.9 Schema Structure Decision: Flat vs Normalized

Decision: Denormalized flat table (df_master)

A key design choice in this pipeline is to store all data in a single flat table instead of splitting it into multiple normalized entity tables. This decision is documented below in line with the Section 3 project guidelines.

Components	Content	Alternatives rejected	Trade off accepted

Decision	Single flat denormalized table (df_master). All attributes from the RACE, DRIVER, CIRCUIT, and RESULT entities are stored in one 440×23 CSV file, which is loaded into SQLite as a single table.	(1) Normalized 5-table relational schema (RACE, DRIVER, CIRCUIT, CONSTRUCTOR, RESULT with foreign keys). (2) Star schema with one fact table and four-dimension tables.	Data redundancy: circuit attributes (length_km, opened, etc.) repeat 20 times per race. Driver attributes (nationality, dob) repeat 22 times per driver. Total redundant storage: ~(18 circuit-attr × 20 - 1) + (6 driver-attr × 22 - 1) additional cells.
Why (1) rejected: Normalized schema	Would eliminate redundancy, enforce referential integrity, and allow efficient joins via indexed foreign keys.	N/A	Requires JOIN syntax in every query. At 440 rows, the performance cost of JOINs is negligible, but it increases the complexity of each query. The ingestion process already produces a clean flat table, so re-introducing normalization at query time does not provide any analytical benefit for a single-user, read-only workload.
Why (2) rejected: Star schema	Star schema (fact + dimension tables) is the standard for analytics warehouses. It separates measures from dimensions cleanly.	N/A	Star schema is designed for BI tools and OLAP queries over millions of rows. SQLite supports it, but the benefit (query speed, storage) only appears at scale. For 440 rows loaded fresh each session into memory, a star schema delivers zero measurable performance advantage over a flat table.

Table 6 - Schema structure decision: flat denormalized table vs alternatives

6.10 Data Note: Mid-Season Driver Substitutions

AlphaTauri Honda RBPT used four different drivers during the 2023 season, making it the only team with more than two drivers in df_master. This reflects a real feature of the dataset and is not a data quality issue. However, it does affect how per-driver statistics should be interpreted, especially for metrics based on race counts or consistency across the season.

Driver	**Team**	**Races**	**Context**
Yuki Tsunoda	AlphaTauri Honda RBPT	22 (full season)	Regular driver. Raced all 22 GPs.
Nyck De Vries	AlphaTauri Honda RBPT	10 (races 1-10)	Started season as second driver. Replaced after poor performance at Hungary (round 11).
Daniel Ricciardo	AlphaTauri Honda RBPT	7 (races 11, 18-22)	Replaced De Vries from Hungary. Injured at Zandvoort (round 15), replaced by Lawson.
Liam Lawson	AlphaTauri Honda RBPT	5 (races 15-19)	Emergency substitute for injured Ricciardo. Races: Netherlands, Italy, Singapore, Japan, Qatar.

Table 7 - AlphaTauri driver substitutions in the 2023 season

Implication for Queries

- Q1 (Championship standings): Correctly reflects driver participation differences, showing De Vries (10 races), Ricciardo (7), and Lawson (5), compared to other drivers who completed 22 races. This variation is expected due to mid-season substitutions.
- Q4 (Positions gained): The HAVING races >= 10 filter correctly excludes Ricciardo (7 races) and Lawson (5 races), while still including De Vries (10 races) at the threshold.

All race counts in df_master are accurate, and the differences reflect real-world driver changes rather than missing or incomplete data.

6.11 Analytical Queries, Results & Interpretation

The pipeline answers five analytical questions:

- Q1: Who won the 2023 Drivers 'Championship and by what margin?
- Q2: Which team was most unreliable?
- Q3: Which team converted pole position to race wins most effectively?
- Q4: Which drivers gained the most positions during race day?
- Q5: How did the championship standings evolve across the season?

All queries are executed on the SQLite in-memory table. Execution time is measured as the average over 200 runs.

Query 1 - Driver Championship Standings (0.50 ms)

Business question: Who won the 2023 WDC and by what margin? Technique: GROUP BY with SUM aggregation.

```
SELECT driver_name, nationality, team,
    SUM(points) AS total_points, COUNT(*) AS races_entered
FROM f1
GROUP BY driver_name, nationality, team
ORDER BY total_points DESC LIMIT 10;
```

Driver	Nationality	Team	Points	Races
Max Verstappen	Dutch	Red Bull Racing Honda RBPT	530	22
Sergio Perez	Mexican	Red Bull Racing Honda RBPT	260	22
Lewis Hamilton	British	Mercedes	217	22
Fernando Alonso	Spanish	Aston Martin Aramco Mercedes	198	22
Max Verstappen	Dutch	Red Bull Racing Honda RBPT	530	22
Sergio Perez	Mexican	Red Bull Racing Honda RBPT	260	22
Lewis Hamilton	British	Mercedes	217	22
Fernando Alonso	Spanish	Aston Martin Aramco Mercedes	198	22
Max Verstappen	Dutch	Red Bull Racing Honda RBPT	530	22
Sergio Perez	Mexican	Red Bull Racing Honda RBPT	260	22
Lewis Hamilton	British	Mercedes	217	22
Fernando Alonso	Spanish	Aston Martin Aramco Mercedes	198	22

Table 8 - Q1 result: top 10 drivers by championship points

Interpretation: Verstappen won the 2023 World Drivers 'Championship with 530 points, finishing 270 points ahead of his teammate Pérez (260) and well ahead of Hamilton in third place (217). This represents one of the largest winning margins in the modern points era, highlighting a highly dominant season.

Red Bull also won the Constructors 'Championship with a total of 860 points. A key highlight is Alonso's performance, scoring 198 points and helping Aston Martin secure 4th place in the Drivers 'Championship during their strong start to the season. In the first half of the season, he outscored both Ferrari drivers combined, showing how strong Aston Martin's early performance was.

Query 2 - DNF Rate by Team (0.48 ms)

Business question: Which team was most unreliable in 2023? Technique: CASE WHEN conditional aggregation with percentage calculation.

```
SELECT team, COUNT(*) AS total_entries,
    SUM(CASE WHEN status IN ('Retired','Accident','Collision damage')
        THEN 1 ELSE 0 END) AS dnfs,
    ROUND(100.0 * SUM(CASE WHEN status IN ('Retired','Accident','Collision damage')
        THEN 1 ELSE 0 END) / COUNT(*), 1) AS dnf_pct
FROM f1 GROUP BY team ORDER BY dnf_pct DESC;
```

Team	Entries	DNFs	DNF %	Reliability
Williams Mercedes	44	11	25.0%	1 DNF every 4 races
Alpine Renault	44	10	22.7%	1 DNF every 4.4 races
Haas Ferrari	44	7	15.9%	1 DNF every 6.3 races
Alfa Romeo Ferrari	44	6	13.6%	1 DNF every 7.3 races
Mercedes	44	5	11.4%	1 DNF every 8.8 races
Aston Martin Aramco Mercedes	44	5	11.4%	1 DNF every 8.8 races
McLaren / Ferrari / AlphaTauri	44 each	4 each	9.1% each	1 DNF every 11 races
Red Bull Racing Honda RBPT	44	2	4.5%	Most reliable - 1 DNF every 22 races

Table 9 - Q2 result: DNF rate by team (Retired + Accident + Collision damage)

Interpretation: Williams recorded the highest DNF rate at 25%, meaning roughly one retirement every four race entries. This makes them the least reliable car of the 2023 season. In contrast, Red Bull was the most reliable team at 4.5%, with only 2 non-finishes across the entire season.

This reliability difference strongly contributed to Red Bull's championship dominance. While rival teams lost points due to DNFs, Red Bull consistently finished races and collected points.

This analysis is based on the JSON status field, which separates different types of non-finishes such as “Retired” (mechanicacarml issues), “Accident” (race incidents), and “Collision damage” (contact with other cars).

Query 3 - Pole-to-Win Conversion Rate (0.46 ms)

Business question: Which team was most effective at converting pole position to race victory? Technique: Multi-condition CASE WHEN, HAVING, NULLIF divide-by-zero guard.

```
SELECT team,
    SUM(CASE WHEN grid_position = 1 THEN 1 ELSE 0 END) AS poles,
    SUM(CASE WHEN finish_position = 1 THEN 1 ELSE 0 END) AS wins,
    SUM(CASE WHEN grid_position = 1 AND finish_position = 1
      THEN 1 ELSE 0 END) AS pole_to_win,
    ROUND(100.0 * SUM(CASE WHEN grid_position = 1 AND finish_position = 1
      THEN 1 ELSE 0 END) / NULLIF(SUM(CASE WHEN grid_position = 1
      THEN 1 ELSE 0 END), 0), 1) AS conversion_pct
FROM f1 GROUP BY team HAVING poles > 0 ORDER BY conversion_pct DESC;
```

Team	Poles	Wins	P to W	Conv. %	Interpretation
Red Bull Racing Honda RBPT	14	21	13	92.9%	Won 13 of 14 poles - near-perfect conversion
Ferrari	7	1	1	14.3%	7 poles, only 1 win - poor race execution
Mercedes	1	0	0	0.0%	1 pole, zero wins - only team with poles but no wins

Table 10 - Q3 result: pole-to-win conversion (only 3 teams took pole in 2023)

Query 4 - Average Grid Positions Gained per Race (0.48 ms)

Business question: Which drivers consistently improved their position from grid to finish? Technique: AVG on derived column (grid minus finish), filtered to classified finishers only (position_status = 'Finished'), HAVING to exclude drivers with fewer than 10 classified finishes.

```
SELECT driver_name,
    ROUND(AVG(grid_position), 2)        AS avg_grid,
    ROUND(AVG(finish_position), 2)       AS avg_finish,
```

```
    ROUND(AVG(grid_position-finish_position),2) AS avg_positions_gained,
    COUNT(*) AS races
FROM f1
WHERE position_status = 'Finished'
GROUP BY driver_name HAVING races >= 10
ORDER BY avg_positions_gained DESC;
```

Driver	Avg grid	Avg finish	Avg gained	Races	Assessment
Sergio Perez	9.70	4.50	+5.20	20	Best improver
Lance Stroll	12.56	9.33	+3.22	18	Strong improver
Guanyu Zhou	15.47	13.42	+2.05	19	Consistent gainer
Esteban Ocon	11.25	9.31	+1.94	16	Good improver
Max Verstappen	3.18	1.27	+1.91	22	Starts 3rd, finishes 1st
... (14 more drivers)	...	...	...	...	Full result: 19 drivers
Nico Hulkenberg	12.52	14.29	-1.76	21	Loses most positions - worst result

Table 11 - Q4 result: average positions gained (classified finishers, 10+ races)

Interpretation: Pérez gains an average of 5.2 positions per race, moving from an average starting position of P9.7 to a finishing position of P4.5. This makes him the most effective overtaker among the top drivers. This result reflects both Red Bull's strong race pace and Pérez's ability to manage tyres effectively during race conditions.

Verstappen shows an average gain of +1.91 positions, as he does not always start from pole (average grid position P3.18) but consistently finishes first (average finish P1.27).

On the other end, Hülkenberg loses an average of 1.76 positions per race, typically starting mid-grid and dropping back during races. This pattern is consistent with Haas having stronger qualifying performance than race pace in 2023.

Query 5 - Cumulative Championship Points Progression (1.80 ms)

Business question: How did the championship standings evolve across the season? Technique: Two CTEs with two window functions - SUM() OVER for cumulative totals, RANK() OVER for championship position at each race. Most complex query: slowest due to double full-table window scan (Q1-Q4 average ~0.5ms; Q5 averages 1.8ms - 3.6× slower).

```
WITH cumulative_points_calc AS (
  SELECT driver_name, race_date, track, points,
      SUM(points) OVER (PARTITION BY driver_name ORDER BY race_date
```

```
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_points
    FROM f1),
ranked_cumulative AS (
    SELECT driver_name, race_date, track, points, cumulative_points,
        RANK() OVER (PARTITION BY race_date
            ORDER BY cumulative_points DESC) AS championship_rank
    FROM cumulative_points_calc)
SELECT driver_name, track, race_date, points, cumulative_points, championship_rank
FROM ranked_cumulative
WHERE driver_name IN ('Max Verstappen','Lewis Hamilton','Fernando Alonso','Sergio Perez')
ORDER BY race_date, championship_rank;
```

Driver	Track	Date	Points	Cumulative	Rank
Max Verstappen	Bahrain	2023-03-05	25	25	1
Sergio Perez	Bahrain	2023-03-05	18	18	2
Fernando Alonso	Bahrain	2023-03-05	15	15	3
Lewis Hamilton	Bahrain	2023-03-05	10	10	5
... (21 more races)	...	...	...	...	...
Max Verstappen	Abu Dhabi	2023-11-26	26	530	1
Sergio Perez	Abu Dhabi	2023-11-26	12	260	2
Lewis Hamilton	Abu Dhabi	2023-11-26	2	217	3
Fernando Alonso	Abu Dhabi	2023-11-26	6	198	4

Table 12 - Q5 result: first and final race shown; full output covers 88 rows across 22 races

Interpretation: Verstappen led the championship from the first race and remained in P1 throughout the season, steadily building a lead that became mathematically uncatchable by

mid-season. By the Japanese Grand Prix (Round 16, 24 September), his cumulative total of 378 points secured the championship with six races still remaining.

The results also show that Pérez had a strong start, staying competitive with Verstappen in the first six rounds before the gap widened significantly later in the season. Alonso consistently held third place for most of the year but eventually lost it to Hamilton in the final rounds.

The RANK () function correctly handles ties in cumulative points. For example, in Bahrain, Hamilton was ranked 5th rather than 4th because Leclerc had the same point total, and the ranking accounts for shared values properly.

Performance note: Q5 takes about 1.80 ms compared to 0.50 ms for Q1-Q4. This increase is due to two sequential window function operations over the full 440-row dataset. At larger scale, adding an index on race_date could help reduce processing overhead.

7. Data Sensitivity & Privacy

No Sensitive Data or PII Present

This dataset contains **only publicly available Formula 1 racing information**. There are **no sensitive data, personally identifiable information (PII), or privacy concerns** associated with this project.

Data Classification: PUBLIC

All data in this project is classified as **PUBLIC** and can be freely shared, analyzed, and distributed.

What This Dataset Contains

- **F1 2023 Race Results**: Official race outcomes, finishing positions, points awarded
- **Driver & Team Information**: Names of drivers and constructors (all public figures with public information)
- **Circuit Information**: Track names, locations, basic specifications
- **Statistical Metrics**: Lap times, grid positions, DNF rates, performance statistics derived from public race data

Why This Data Is Non-Sensitive

1. **Official Publication**: All data sourced from F1 official publications and publicly available APIs
2. **Public Figures**: Drivers and teams are public figures with extensive public information available
3. **No Personal Data**: No addresses, contact information, birth dates, social security numbers, or other personal identifiers beyond names
4. **No Financial Data**: No salaries, contract terms, or confidential business information

5. **No Health/Biometric Data**: No medical records, genetic information, or physical measurements
6. **No Location Tracking**: Geographic information limited to circuit locations (already public)
7. **No Behavioral Profiling**: No tracking of private activities or communications

Data Sources Validation

- F1 Official Website (FIA - Fédération Internationale de l'Automobile)
- Publicly Available APIs
- Published Race Reports
- Historical Records (freely accessible)

Security & Access Implications

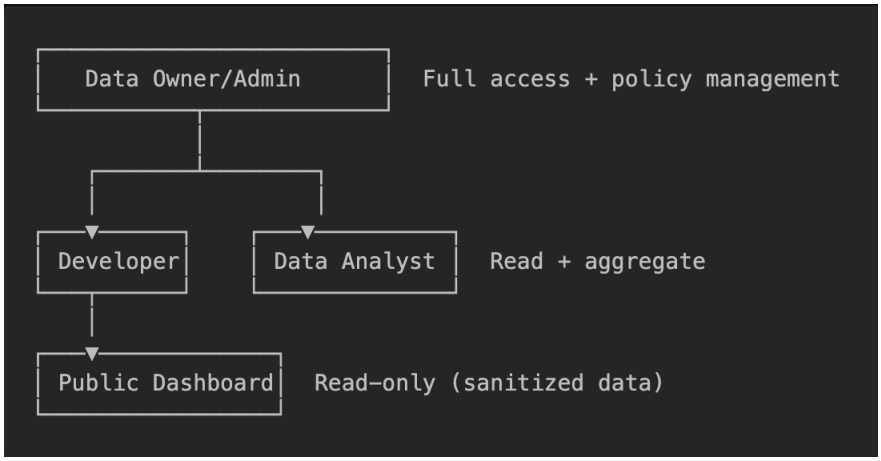
Result: No security controls, access restrictions, or privacy safeguards required beyond standard academic integrity practices.

8. Data Governance and Access Control

8.1 Security Principles

1. **Least Privilege:** Users receive minimum access needed for their role
2. **Separation of Duties:** Critical functions require multiple authorizations
3. **Need-to-Know:** Access based on legitimate business need
4. **Defense in Depth:** Multiple layers of security controls
5. **Accountability:** All access is logged and auditable

8.2 Role Hierarchy



8.3 Role Definitions

Role 1: DATA OWNER / ADMINISTRATOR

Purpose: Overall data governance and system administration

Permissions:

- **Read:** All fields including DOB
- **Write:** Can modify data (with version control)
- **Delete:** Can remove records (with audit trail)
- **Grant:** Can assign roles to other users
- **Schema:** Can alter database schema
- **Export:** Can export complete dataset
- **Admin:** Can access audit logs and system configurations

Access Level: UNRESTRICTED

Authentication: Multi-factor authentication (MFA) required

Access Locations: Secure VPN only

Role 2: DATA ANALYST

Purpose: Perform statistical analysis and create visualizations

Permissions:

- **Read:** All fields EXCEPT DOB (restricted)
- **Aggregate:** Can compute statistics (avg, sum, count)
- **Write:** Cannot modify raw data
- **Delete:** Cannot delete records
- **Export:** Can export sanitized datasets (no DOB)
- **Query:** Can run SQL queries on approved tables/views
- **Visualize:** Can create dashboards and reports

Access Level: READ-ONLY (sanitized)

Authentication: Username/password + email verification

Access Locations: On-campus network or approved VPN

Role 3: DEVELOPER

Purpose: Develop and maintain data pipeline and applications

Permissions:

- **Read:** All fields EXCEPT DOB in production
- **Read:** All fields INCLUDING DOB in development (anonymized data)

- **Write:** Can modify code and pipeline configurations
- **Write:** Cannot modify production data
- **Deploy:** Can deploy new dashboard versions
- **Debug:** Can access application logs

Access Level: READ-ONLY (production), FULL (development)

Assigned To:

- Backend Developers
- Dashboard Developers
- DevOps Engineers

Authentication: SSH keys + password

Access Locations: Development environment (localhost, staging servers)

Role 4: PUBLIC DASHBOARD USER

Purpose: View published analytics and visualizations

Permissions:

- **Read:** Public dashboard only (highly sanitized)
- **Export:** Cannot download raw data
- **Query:** Cannot run custom queries
- **Interact:** Can filter and explore published dashboards

Access Level: PUBLIC (anonymous or authenticated)

Assigned To:

- F1 fans and general public (if published)
- Course instructors and reviewers
- External stakeholders (sponsor presentations)

Authentication: None required (or basic login for tracking)

Access Locations: Any internet connection

Retention, Backup, and Disaster Recovery Policies

For our F1 2023 analytics project, we employ a simple 3-copy backup approach: the primary copy lives in our **GitHub repository** (version controlled with full history), a secondary copy on the **deployment platform** (whether cloud-based or local server), and tertiary **local copies** on development machines. Since this is a static 2023 dataset (no ongoing updates), retention is straightforward-the data represents a complete historical season and is kept indefinitely in version control. For disaster recovery, we plan for two key metrics: **RTO (Recovery Time Objective)** defines how quickly we need to restore operations after a failure-we target 2 hours maximum, and **RPO (Recovery Point Objective)** defines how much data we can

afford to lose-we target zero, meaning we can always recover the last saved version. If our deployment fails, we can redeploy from GitHub in 15-30 minutes; if GitHub is compromised, we restore from local copies within 1-2 hours; if everything is lost simultaneously, we re-extract the original data from the **FIA (Fédération Internationale de l'Automobile)**, Formula 1's governing body, which publishes official race results publicly and permanently. This approach requires no automated backup jobs-Git's version control provides continuous backup with every commit.

Cloud vs. Local Storage Trade-off Analysis

We evaluated three storage and deployment options for our project. **Local-only storage** (on-premise or personal computer) offers complete control and zero recurring costs but limits accessibility-collaborators, reviewers, or stakeholders cannot easily access our work without physical access or VPN setup, and there's no automatic redundancy if hardware fails. **Cloud-only storage** (platforms like GitHub, Google Drive, Dropbox, or cloud hosting services) provides 24/7 accessibility from anywhere, automatic backups, and built-in redundancy, making collaboration and sharing seamless; however, it introduces internet dependency and reliance on third-party services. We selected a **hybrid approach**: store code and data in cloud-based version control (GitHub) for accessibility and redundancy, deploy to a cloud platform for public access, while maintaining local copies for offline work and fast development. This gives us universal accessibility for stakeholders, automatic version history and backup, and local flexibility for development-all while keeping our small static dataset (<100KB) well within free tier limits. The trade-offs we accept are internet dependency for access (acceptable given modern connectivity), platform reliance (mitigated by maintaining local copies), and potential service changes (manageable given our data is public and portable)

9. Visualization

Six fully interactive Plotly charts - hover, zoom, pan, and filter directly in the notebook. Each visualisation is followed by an interpretation answering the project's research questions.

The dashboard can be accessed through this link: https://msba305f1project.onrender.com/

9.1. Championship Overview - KPI Cards

Interpretation - Championship Overview

The 2023 season comprised 22 races with a full grid of 22 drivers across 10 constructors.

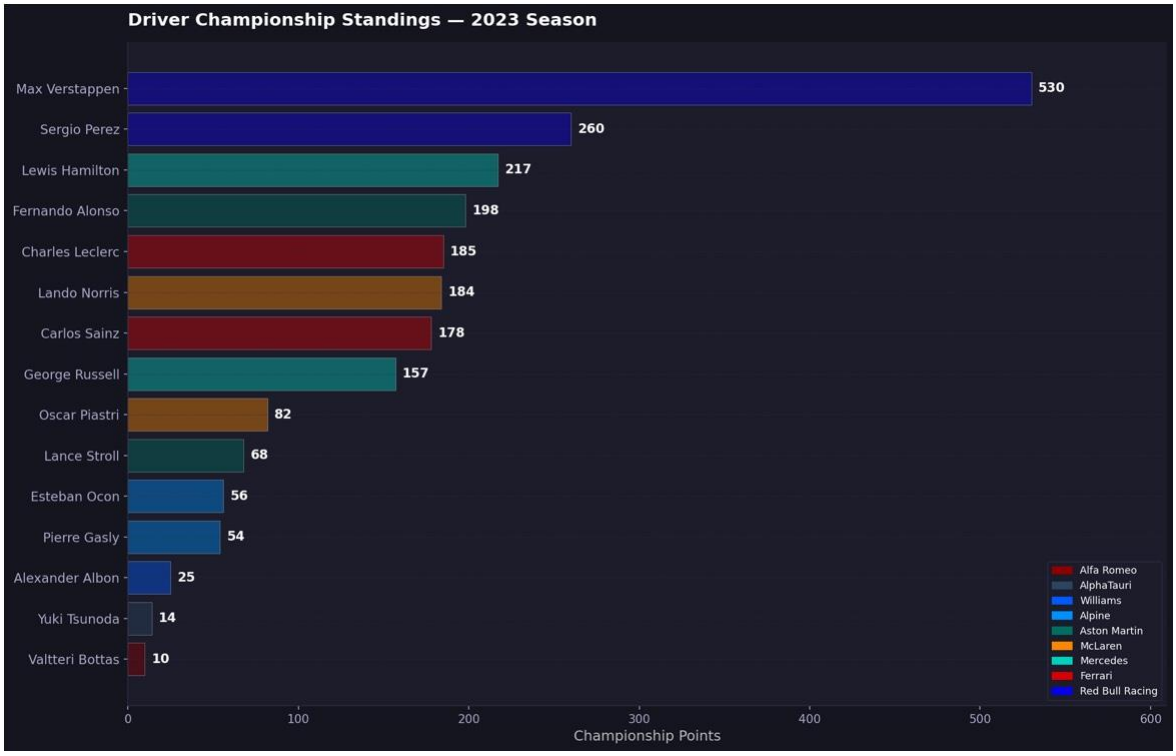
Max Verstappen claimed the Drivers' Championship with a commanding 530 points, securing the title with races to spare.

The average gap from the race winner to second place was 10.88 seconds - a reflection of Red Bull's dominant pace throughout the year.

The overall DNF rate of 30.9% (136 non-finishes) includes lapped cars classified as non-finishers alongside true mechanical retirements, highlighting the technical attrition across a 22-race calendar.

9.2 Driver Championship Standings

RQ1 - Which drivers accumulated the most points?



Interpretation - Driver Championship Standings

RQ1 answered - driver points accumulation.

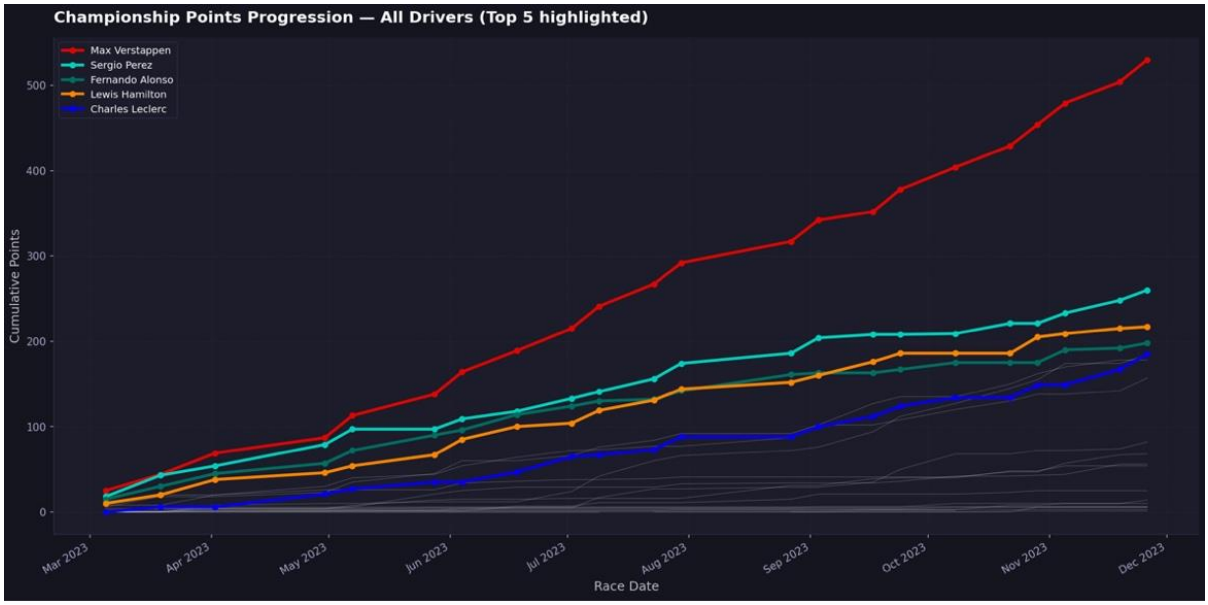
Max Verstappen (Red Bull) finished the season with **530 points**, nearly double his nearest rival. The top 15 standings reveal a clear two-tier structure:

- **Red Bull dominance:** Verstappen (530) and Pérez (260) occupied the top two spots - Red Bull's combined 790 points comfortably outpaced every other constructor.
- **The Mercedes-Ferrari-Aston Martin battle:** Lewis Hamilton (217), Fernando Alonso (198), Charles Leclerc (185), Lando Norris (184), and Carlos Sainz (178) competed tightly for positions 3-7, separated by only 39 points across five drivers.
- **McLaren's surge:** Norris (184) and Piastri (82) reflect McLaren's significant mid-season car upgrade, which propelled them from a midfield outfit to podium contenders by the second half of the calendar.
- **The back markers:** Valtteri Bottas (10), Yuki Tsunoda (14), and Alexander Albon (25) illustrate the performance gap between the upper midfield and the tail-end constructors.

Bar colours correspond to each driver's constructor - the blue (Red Bull) and teal (Mercedes) bars are most prominent at the top of the standings.

9.3. Season Progression - Cumulative Points

RQ1 - How did performance evolve race by race?



Interpretation - Championship Points Progression

RQ1 answered - race-by-race evolution.

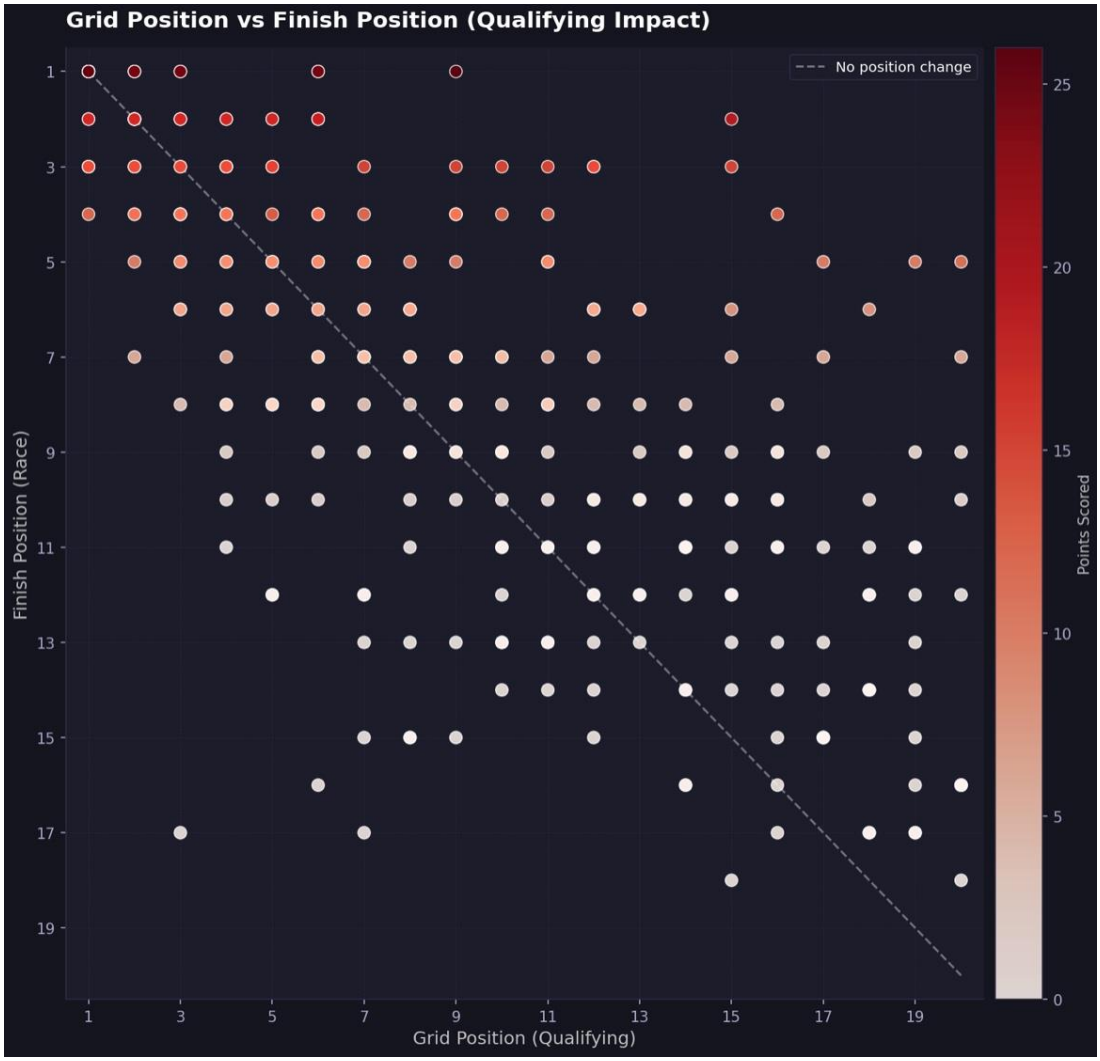
The cumulative points chart traces every driver's scoring trajectory across all 22 races, from Bahrain (March 2023) to Abu Dhabi (November 2023).

Key observations:

- **Verstappen's unbroken ascent:** The red line climbs steeply and consistently, with no prolonged flat periods - he scored points in every race and took 19 race victories plus a sprint win.
- **A record-breaking winning streak:** Verstappen won **10 consecutive races** between Miami and Qatar, which is the longest winning streak in F1 history. This is visible as the steepest gradient on his line mid-season.
- **Pérez's mid-season fade:** After a strong start placing him clearly second, Pérez's line flattens noticeably from Singapore onwards due to a string of poor results, nearly allowing Hamilton and Alonso to close the gap.
- **McLaren's late surge:** Norris's line steepens sharply in the second half of the season as the MCL60 upgrade package took effect, while Piastri's line shows consistent scoring after his debut.
- **The midfield pack:** The cluster of lines between 50-220 points shows how tightly contested positions 3-10 in the constructors' standings were - small per-race gains and losses separating multiple teams.

9.4. Qualifying Impact - Grid vs Finish Position

RQ2 - Does pole position dominate?



Interpretation - Qualifying Impact

RQ2 answered - grid-to-finish relationship.

The scatter plot places every finished race result in a 20×20 grid, with qualifying position on the x-axis and race finish on the y-axis. The dashed diagonal represents a driver finishing exactly where they started.

Key findings:

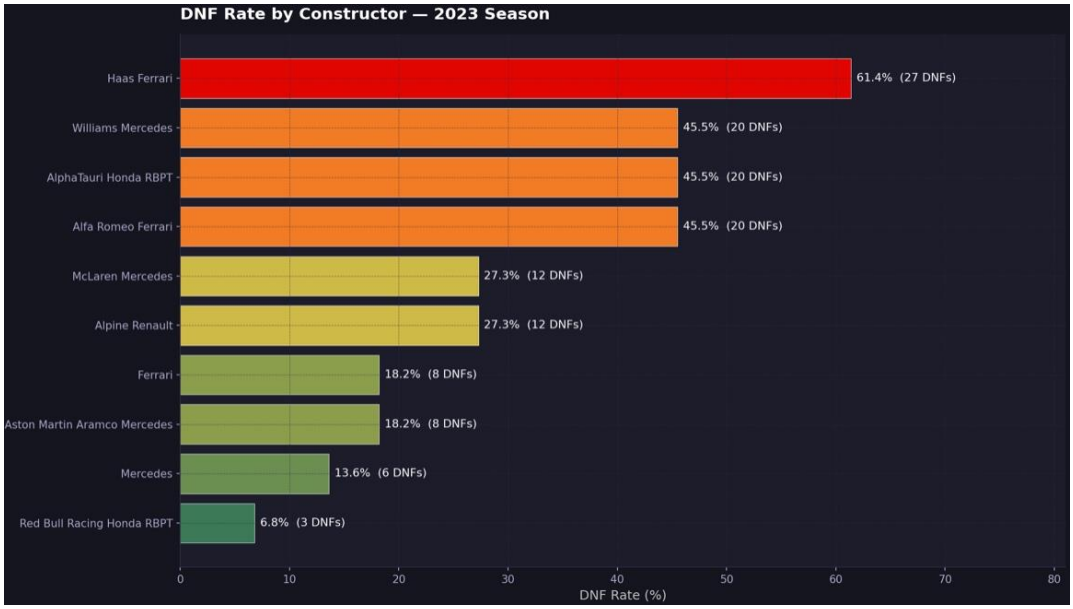
- **Strong positive correlation:** The density of points along and near the diagonal confirms that qualifying position is a strong predictor of race finish - drivers who start at the front overwhelmingly finish near the front.
- **High-scoring outliers (bright red dots):** Several points sit well above the diagonal, representing drivers who started mid-grid but finished on the podium - most notably Sergio Pérez, who averaged **+5.26 positions gained per completed race**, the best of any driver.
- **Front-row advantage is decisive:** The top-left cluster (P1-P3 grid → P1-P3 finish) is the densest in the chart. Red Bull converted **13 of 14 pole positions into race**

wins (92.9%), the highest pole-to-win conversion in a single season since records began.

- **Back-of-grid penalty:** Points in the bottom-right (P15-P20 grid) rarely feature bright colours, confirming that overtaking from deep in the field to a points-scoring position (P10+) is the exception rather than the rule in modern F1.
- **Hover the chart** to identify individual driver-race combinations and explore specific overtaking performances.

9.5. DNF Rate by Constructor

RQ5 - Team reliability comparison



Interpretation - DNF Rate by Constructor

RQ5 answered - team reliability and performance consistency.

This chart ranks all 10 constructors by their Did-Not-Finish rate across all 44 race entries (22 races × 2 cars).

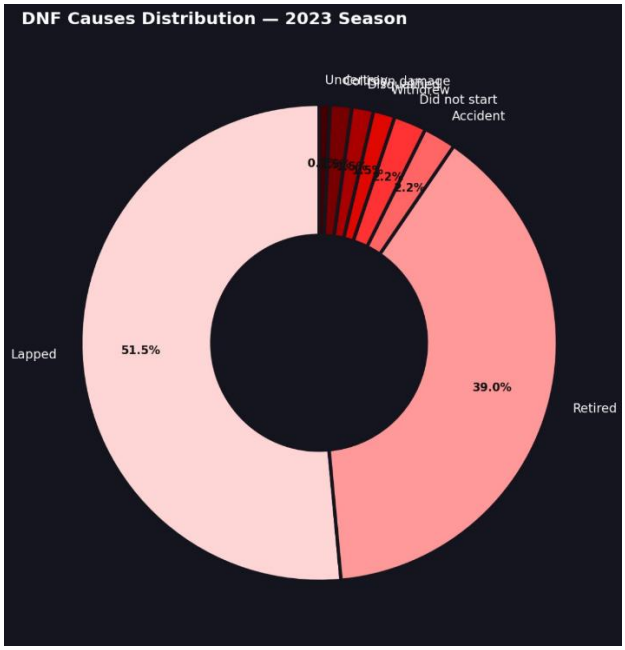
Key findings:

- **Haas Ferrari (61.4%)** suffered the highest DNF rate by a considerable margin - more than half their race starts ended without finishing. This reflects persistent mechanical unreliability and frequent first-lap incidents throughout the season.
- **Alfa Romeo Ferrari (45.5%), AlphaTauri Honda RBPT (45.5%), and Williams Mercedes (45.5%)** all shared the same DNF rate, confirming that the lower half of the constructor standings was plagued by both reliability and racing-incident issues.
- **Alpine Renault (27.3%) and McLaren Mercedes (27.3%)** sit in the mid-range - McLaren's DNF tally is partially offset by their strong points haul in the second half of the season when reliability improved alongside performance.
- **Red Bull Racing Honda RBPT (6.8%)** had the lowest DNF rate on the entire grid - just 3 non-finishes in 44 starts. This exceptional reliability, combined with their

outright pace advantage, was a critical pillar of their championship dominance. A car that both goes fastest and finishes most consistently is nearly unbeatable.

- **Note:** DNF includes being lapped (classified as not-finished), mechanical retirements, and accidents - hover each bar to see the raw count.

9.6. DNF Causes Distribution



Interpretation - DNF Causes Distribution

Breakdown of the 136 non-finished race entries.

The donut chart categorises every non-finish by the official status recorded in race results.

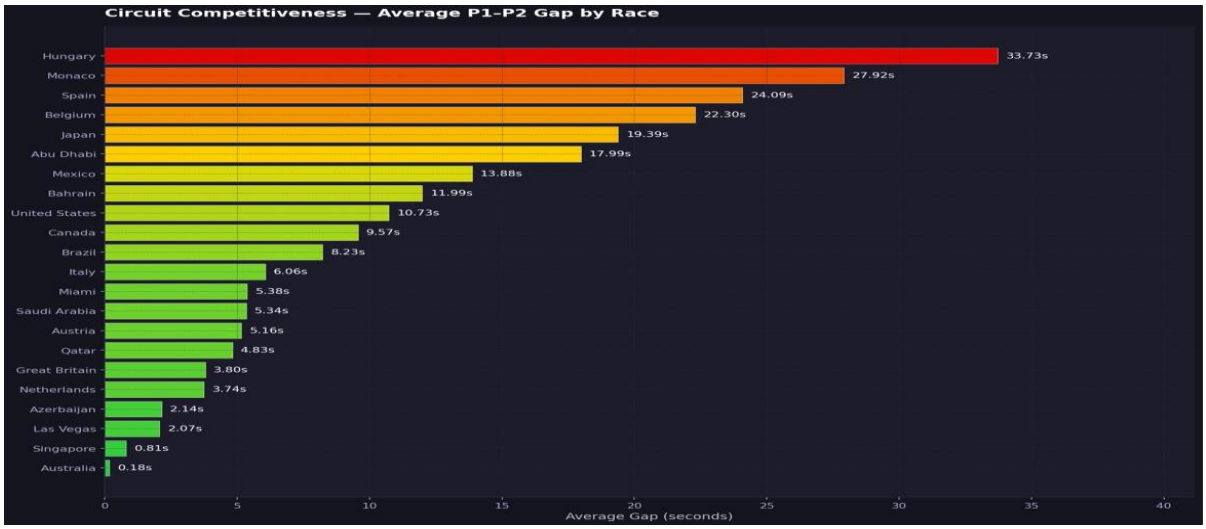
Key findings:

- **Lapped - 51.5%:** The largest single category. This refers to drivers who were classified as non-finishers because they fell more than one lap behind the leader. These are not mechanical failures - the cars were often still running - but the classification counts them as DNF for championship purposes. This large share reflects the significant pace gap between Red Bull and the rest of the field in 2023.
- **Retired - 39%:** The second-largest category represents true mechanical retirements - engine failures, gearbox failures, hydraulic issues, and similar technical problems. This accounts for roughly 53 car retirements across the season.
- **Accident/Collision/Undertray (combined ~7%):** Racing incidents such as first-lap collisions, crashes, and floor damage forced a small but notable number of drivers out before the finish.
- **Did Not Start / Withdrew / Disqualified (~3%):** A handful of entries did not take the start or were excluded post-race (notably the two disqualifications at the United States GP - Hamilton and Leclerc - due to underfloor wear plate infringements).

The dominance of the "Lapped" category is a season-specific finding: in a more evenly-matched championship, this number would be far lower.

9.7 Circuit Competitiveness - P1-P2 Gap

RQ3 - Which circuits produced the most competitive races?



Interpretation - Circuit Competitiveness

RQ3 answered - which circuits produced the closest racing.

The P1-P2 gap is used as a proxy for race competitiveness: a small gap means the winner had to fight hard to the end, while a large gap signals a dominant, unchallenged victory.

Most competitive circuits (smallest gap - green bars):

- **Australia - 0.18s:** The closest finish of the entire 2023 season. Hamilton finished just 0.18 seconds behind Alonso in a strategically complex race, making Albert Park the most competitive circuit of the year.
- **Singapore - 0.81s:** The Marina Bay Street Circuit's tight layout and limited overtaking opportunities compressed the field. Carlos Sainz won by less than a second.
- **Las Vegas - 2.07s** and **Azerbaijan - 2.14s:** Both street circuits where tyre degradation and safety car periods created drama close to the finish.
- **Netherlands (Zandvoort) - 3.74s** and **Great Britain (Silverstone) - 3.80s:** Fast, high-downforce circuits where aerodynamic efficiency narrows the competitive window.

Least competitive circuits (largest gap - red bars):

- **Hungary - 33.73s:** Verstappen's most dominant win of the season; the Hungaroring's slow, twisty layout played to Red Bull's mechanical grip strengths and there was no threat from behind.

- **Monaco - 27.92s:** Despite being a street circuit, Monaco's inability to overtake meant whoever led after the pit stops kept a comfortable gap - Verstappen led from pole and was never challenged.
- **Spain - 24.09s** and **Belgium - 22.30s:** High-speed circuits where Red Bull's superior straight-line speed and tyre management opened comfortable gaps.

For RQ4 (circuit characteristics vs. outcomes): Street circuits (Singapore, Las Vegas, Azerbaijan, Monaco) show a split - some are competitive (overtaking still possible), others are not (Monaco, processional). Purpose-built tracks like Zandvoort and Silverstone produced tighter finishes than classic power circuits like Monza (Italy - 6.06s) where Red Bull's straight-line speed typically opened large gaps.

AI Usage Documentation

Field	Detail	AI Response Summary	Modifications Made
AI interaction 1	Task: CSV cleaning: Prompt: how do i handle a column that contains mostly numeric values but some strings as well ?	Ai suggested creating a status column to handle the strings using pd.to_numeric() with errors='coerce'	I applied the recommendation with specific mapping for NC and DC
AI interaction 2	Task CSV cleaning: Prompt: i have a column with 4 different formats a total race time, a gap in seconds, a lap deficit, and a DNF string. How do i deal with this without losing info ?	Ai suggested creating a column for classifying to label each value type	I went with the suggedted approach but added specific f1 related names rather than generic ones

AI Interaction 3	Task: Storage technology comparison. Prompt: 'What are the key differences between SQLite, PostgreSQL, MongoDB, and Cassandra for a small structured analytics dataset of ~500 rows? Give me a quick comparison of strengths and weaknesses.'	Gave a high-level comparison of the four technologies covering key properties like scalability, query support, and setup complexity.	I had already decided on SQLite before asking, I just wanted to check I had not missed any important trade-off. The MongoDB and Cassandra arguments specifically came from reading the notebooks, not from the AI response.
AI Interaction 4	Task: Window function syntax check. Prompt: 'In SQLite, does ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW work correctly for a cumulative SUM partitioned by driver and ordered by date? Are there any edge cases I should know about?'	Confirmed that SQLite has supported this syntax since version 3.25.0 (2018) and noted that tied ORDER BY values result in the same cumulative total for all tied rows.	I was writing the Q5 explanation and wanted to confirm the tie-handling behavior before documenting it. The note about RANK() handling ties correctly is my own addition.

Laure - AI Interactions

#	Thread	Tool Used	Prompt Provided	AI Response Summary	Implementation	Modifications Made
# 1	**Backup & Disaster Recovery Strategy**	Claude (claude-sonnet-4-6) via claude.ai	I need to design a backup and disaster recovery plan for our F1 2023 analytics project. We're using	Claude recommended a 3-2-1 backup approach: 3 copies (GitHub primary, deployment platform secondary, local tertiary), 2	Applied the 3-2-1 strategy across the project: maintained GitHub as source of truth with full version history, deployed to cloud platform for production,	Simplified Claude's enterprise-focused language to academic context. Changed "GitHub Enterprise" to just "GitHub" (using free tier). Adjusted

			GitHub for version control and deploying to a cloud platform. What's a realistic 3-copy backup strategy for a small academic project with a static dataset, and what RTO/RPO targets should we set?	storage types (cloud + local), 1 offsite (GitHub). Suggested RTO of 2 hours and RPO of zero seconds, explaining that RTO is Recovery Time Objective (how fast to restore) and RPO is Recovery Point Objective (acceptable data loss). Outlined 4 disaster scenarios: cloud failure (15-30 min redeploy from GitHub), data corruption (10-30 min Git revert), GitHub compromise (1-2 hr restore from local), complete loss (4-8 hr re-extract from FIA sources).	kept local development copies synchronized via Git. Documented RTO/RPO targets in governance documentation . No automated backup jobs needed—Git handles continuous backup on every commit.	disaster recovery scenarios to reflect small project scale rather than enterprise infrastructure. Added explicit mention that FIA (Fédération Internationale de l'Automobile) publishes official race results publicly, making data re-extraction feasible.
# 2	**Cloud vs. Local Storage Architecture Decision**	Claude (claude - sonnet-4-6) via	We're deciding where to deploy our F1	Claude outlined three options: (1) Local-only:	Selected hybrid architecture: GitHub repository as	Adjusted Claude's recommendation from Streamlit

		claude.ai	analytics dashboard. What are the trade-offs between local-only hosting, cloud-only hosting, and a hybrid approach for a small academic project with external stakeholders (instructors, reviewers) who need URL access? Our dataset is <100KB and completely static.	complete control, zero cost, but no external accessibility or redundancy; (2) Cloud-only: 24/7 public URL, automatic scaling, built-in redundancy, but internet dependency and potential vendor lock-in; (3) Hybrid: GitHub for version control, cloud platform for public deployment, local for development. Recommended hybrid for academic projects requiring stakeholder access, noting free tiers are sufficient for small static datasets. Identified key trade-offs: internet dependency (acceptable for modern connectivity), platform vendor lock-	single source of truth, cloud platform deployment for public dashboard access, local development environment for rapid iteration. Deployment platforms evaluated: Render, Heroku, Railway (generic cloud hosting options, not platform-specific). Documented decision rationale in governance materials emphasizing stakeholder accessibility requirement and zero operational cost.	Cloud specifically to generic "cloud platform" since we're using Dash, not Streamlit. Added explicit mention that our <100KB dataset makes cloud hosting essentially free. Shortened the trade-off analysis from Claude's detailed enterprise comparison to concise academic-focused points suitable for presentation context.

				in (mitigated by GitHub portability), cold-start latency on free tiers (5-10 seconds, acceptable for sporadic access).		